```
step | byte lobits                                    "dispatch on next code syllable"
    [byte ← self nextbyte.
    lobits ← byte|16.
    byte/16=1⇒[self push: receiver instfield: lobits];          "load from instance"
           =2⇒[self push: tempframe·lobits];                    "load from temps (and args)"
           =3⇒[self push: (method literals: lobits)];           "load from literals"
           =4⇒[self push: (method literals: lobits) value];     "load indirectly from literals"
           =5⇒[self push: self instfield: lobits];              "load from this Context"
           =6⇒[self push: ☞("1 0 1 2 10 true false nil)·lobits]; "frequent constants"
           =7⇒[⇑self send: (method literal: lobits)];
           =8⇒[⇑self send: (SpecialMessages·lobits)];           "frequent messages"
           =9⇒[lobits<8⇒[pc← pc+lobits]                         "short jump forward"
                 self pop⇒[] pc← pc+lobits-8];                  "short branch if false and pop"
           =10⇒[lobits<8⇒[pc← lobits-3*256+self nextbyte+pc]"long jump forward and back"
                 self pop⇒[pc← pc+1];                         "skip extension byte on true"
                 pc← lobits-11*256+self nextbyte+pc];         "long bfp"
           =11⇒[lobits=0⇒[self pop];                          "pop stack"
                 =1⇒[self store: self top into: self nextbyte]; "store"
                 =2⇒[self store: self pop into: self nextbyte]; "store and pop"
                 =3⇒[sender push: self top. ⇑sender]]         "return value to sender"
    ]
```